

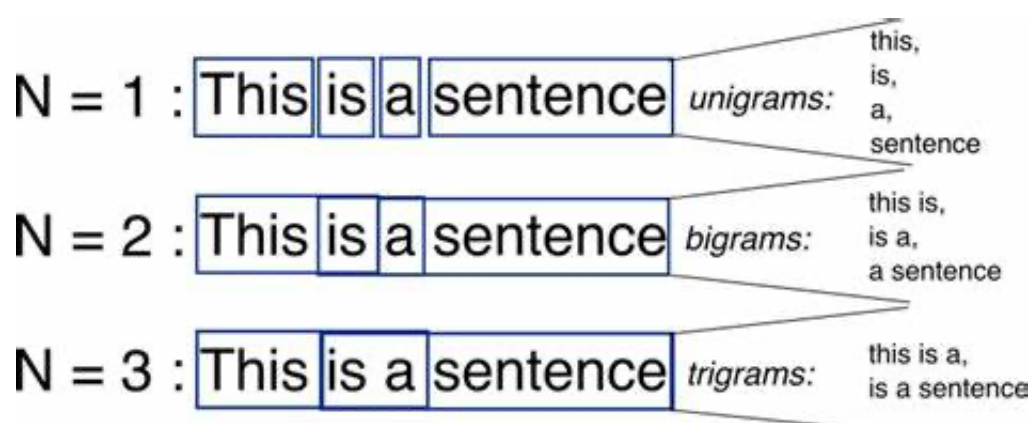
Name: Mohammed Feras Sawab

ID: 2240009066

N-Grams

It's a probabilistic model that's trained on a corpus of text. Such a model is useful in many NLP applications including speech recognition, machine translation and predictive text input. An N-gram model is built by counting how often word sequences occur in corpus text and then estimating the probabilities.

Types of N-Grams



Use-cases of N-Grams

- Auto completion of sentences
- Auto spell check
- Voice-based personal assistant bots

Unigram

Unigram an n-gram consisting of a single item from a sequence.

- It is commonly used to calculate the probability of finding a word in an article.

$$\text{countof}(A) \div \text{corpuslength}$$

```
In [1]: from nltk.util import ngrams
        from nltk.tokenize import word_tokenize

        string = "moses supposes his toeses are roses but moses supposes er
        unigrams = ngrams(word_tokenize(string), 1)

        for item in unigrams:
            print(item)

('moses',)
('supposes',)
('his',)
('toeses',)
('are',)
('roses',)
('but',)
('moses',)
('supposes',)
('erroneously',)
```

Calculating probabilities in unigram models

```
In [2]: ### Calculate counts of each word

        unique_string = set(word_tokenize(string))

        words_frq = []
        for uniques in unique_string:
            frq = string.count(uniques)
            words_frq.append((uniques, frq))

        words_frq
```

```
Out[2]: [('supposes', 2),
          ('roses', 1),
          ('moses', 2),
          ('but', 1),
          ('are', 1),
          ('erroneously', 1),
          ('his', 1),
          ('toeses', 1)]
```

```
In [3]: ### Calculate probabilities:

        probab = []

        for count in range(len(words_frq)):
            pro = words_frq[count][1]/len(string)
            probab.append(pro)

        print('the probabilities are: ', probab)
```

the probabilities are: [0.0303030303030304, 0.0151515151515152, 0.0303030303030304, 0.0151515151515152, 0.0151515151515152, 0.0151515151515152, 0.0151515151515152, 0.0151515151515152]

```
In [4]: probabilities = []
        for i,j in zip(unique_string, probab):
            probabilities.append((i,j))
        probabilities
```

```
Out[4]: [('supposes', 0.0303030303030304),
         ('roses', 0.0151515151515152),
         ('moses', 0.0303030303030304),
         ('but', 0.0151515151515152),
         ('are', 0.0151515151515152),
         ('erroneously', 0.0151515151515152),
         ('his', 0.0151515151515152),
         ('toeses', 0.0151515151515152)]
```

Bigram

Bigram an n-gram consisting of two items from a sequence

- Two words that are used together to mean something specific

$$\text{countof}(A \text{ then } B) \div \text{countof}(A)$$

Method 1

```
In [5]: import nltk as nl
        from nltk.tokenize import word_tokenize

        string = "moses supposes his toeses are roses but moses supposes er

        bigrams = nl.bigrams(word_tokenize(string))
        for i in bigrams:
            print(i)

('moses', 'supposes')
('supposes', 'his')
('his', 'toeses')
('toeses', 'are')
('are', 'roses')
('roses', 'but')
('but', 'moses')
('moses', 'supposes')
('supposes', 'erroneously')
```

Method 2

```
In [6]: from nltk.util import ngrams
        from nltk.tokenize import word_tokenize

        string = "moses supposes his toeses are roses but moses supposes er
```

```
bigrams = ngrams(word_tokenize(string),2)

for i in bigrams:
    print(i)
```

```
('moses', 'supposes')
('supposes', 'his')
('his', 'toeses')
('toeses', 'are')
('are', 'roses')
('roses', 'but')
('but', 'moses')
('moses', 'supposes')
('supposes', 'erroneously')
```

Trigram

Trigram an n-gram consisting of three items of a sequence

- Three words that are used together to mean something specific

Method 1

```
In [7]: import nltk as nl
        from nltk.tokenize import word_tokenize

        string = "moses supposes his toeses are roses but moses supposes er
        trigrams = nl.trigrams(word_tokenize(string))

        for i in trigrams:
            print(i)
```

```
('moses', 'supposes', 'his')
('supposes', 'his', 'toeses')
('his', 'toeses', 'are')
('toeses', 'are', 'roses')
('are', 'roses', 'but')
('roses', 'but', 'moses')
('but', 'moses', 'supposes')
('moses', 'supposes', 'erroneously')
```

Method 2

```
In [8]: from nltk.util import ngrams
        from nltk.tokenize import word_tokenize

        string = "moses supposes his toeses are roses but moses supposes er
        trigrams = ngrams(word_tokenize(string),3)

        for i in trigrams:
```

```
print(i)
('moses', 'supposes', 'his')
('supposes', 'his', 'toeses')
('his', 'toeses', 'are')
('toeses', 'are', 'roses')
('are', 'roses', 'but')
('roses', 'but', 'moses')
('but', 'moses', 'supposes')
('moses', 'supposes', 'erroneously')
```

What is padding in NLP?

padding is a technique used to ensure that all input sequences have the same length. This is necessary because neural networks require inputs that have the same shape and size. However, when we pre-process and use texts as inputs for our model, not all sentences have the same length. In other words, some sentences are longer or shorter than others. We need to have inputs with the same size, and this is where padding comes in 1.

Padding is a special form of masking where the masked steps are at the start or the end of a sequence. Padding comes from the need to encode sequence data into contiguous batches: in order to make all sequences in a batch fit a given standard length. The padding is added before splitting the sentence.

n-1 padding is added.

```
In [9]: from nltk.util import ngrams
        from nltk.tokenize import word_tokenize

        string = "moses supposes his toeses are roses but moses supposes er
        list(ngrams(word_tokenize(string), pad_left=True, left_pad_symbol="<s>".
                pad_right=True, right_pad_symbol="</s>"))
```

```
Out[9]: [('<s>', 'moses'),
         ('moses', 'supposes'),
         ('supposes', 'his'),
         ('his', 'toeses'),
         ('toeses', 'are'),
         ('are', 'roses'),
         ('roses', 'but'),
         ('but', 'moses'),
         ('moses', 'supposes'),
         ('supposes', 'erroneously'),
         ('erroneously', '</s>')]
```

Alternatively, We can get rid of the parameters by using

```
pad_both_ends method in NLTK
```

Note the n argument, that tells the function we need padding for bigrams.

```
In [10]: from nltk.lm.preprocessing import pad_both_ends
         from nltk.util import ngrams
         from nltk.tokenize import word_tokenize

         string = "moses supposes his toeses are roses but moses supposes er
         list(ngrams(pad_both_ends(word_tokenize(string), n=2), n=2))

Out[10]: [('<s>', 'moses'),
          ('moses', 'supposes'),
          ('supposes', 'his'),
          ('his', 'toeses'),
          ('toeses', 'are'),
          ('are', 'roses'),
          ('roses', 'but'),
          ('but', 'moses'),
          ('moses', 'supposes'),
          ('supposes', 'erroneously'),
          ('erroneously', '</s>')]
```

Training the Maximum Likelihood Model

Maximum likelihood estimation estimates the model parameters such that the probability is maximized.

- Get the requisite n_grams frequency counts from a corpus.
- Normalize them to a 0-1 range.
- Build the vocabulary: to create this vocabulary we need to pad our sentences (just like for counting ngrams) and then combine the sentences into one flat stream of words.

To do the above steps, we need to create and Ngram and add the padding then flatten the list. All these steps are requested for pre-processing to train the MLE model. However, we can use `padded_everygram_pipeline()` in NLTK to do all the steps in one call.

References:

<https://www.nltk.org/api/nltk.lm.html>

<https://www.kaggle.com/code/alvations/n-gram-language-model-with-nltk>

```
In [11]: from nltk.lm.preprocessing import padded_everygram_pipeline
         from nltk.util import ngrams
         from nltk.tokenize import word_tokenize
         from nltk.lm import smoothing
```

```
string = "moses supposes his toeses are roses but moses supposes er

#Create Bigram Language Model
train, vocab = padded_everygram_pipeline(2, [word_tokenize(string)])
```

```
In [12]: from nltk.lm import MLE
lm = MLE(2)
lm.fit(train, vocab) #Train the Maximum Likelihood Model
print(lm.vocab)
print(len(lm.vocab))

<Vocabulary with cutoff=1 unk_label='<UNK>' and 11 items>
11
```

```
In [13]: lm.vocab.lookup(['moses', 'supposes'])
```

```
Out[13]: ('moses', 'supposes')
```

```
In [14]: lm.vocab.lookup(['moses'])
```

```
Out[14]: ('moses',)
```

counts for unigrams and bigrams

```
In [15]: print(lm.counts['moses'])
# a then b
print(lm.counts[['moses']][['supposes']])
```

```
2
2
```

```
In [16]: print(lm.score('moses')) #Calculate Bigram Probabilities
```

```
0.16666666666666666
```

Here's how you get the score for a word given some preceding context. For example we want to know what is the chance that "b" is preceded by "a".

```
In [17]: # b then a
lm.score("supposes", ["moses"])
```

```
Out[17]: 1.0
```

Evaluating n_grams models using perplexity

Preplexity is the probability of the test set normalized by the number of words

```
In [18]: test = [('moses', 'supposes'), ('toeses', 'are')]
print(lm.perplexity(test))
```

```
1.0
```

Part 1 Task#1: Generating tweets using bigrams

In this Task, we will build a bigram MLE model to generate tweets.

The agenda is:

1. load the data: <https://www.kaggle.com/datasets/adizafar/large-random-tweets-from-pakistan>
2. pre-processing the data:
 - Remove hashtags
 - Remove RT
 - Remove any website
 - Remove any mentions
3. build a bigram MLE model
4. evaluate the model

before we start we need to install library to handle the emojis in tweets

```
pip install emoji
```

```
In [19]: !pip install emoji
```

```
WARNING: Ignoring invalid distribution ~cipy (/Users/firas/miniconda3/lib/python3.11/site-packages)
Requirement already satisfied: emoji in /Users/firas/miniconda3/lib/python3.11/site-packages (2.15.0)
WARNING: Ignoring invalid distribution ~cipy (/Users/firas/miniconda3/lib/python3.11/site-packages)
WARNING: Ignoring invalid distribution ~cipy (/Users/firas/miniconda3/lib/python3.11/site-packages)
```

```
In [20]: import glob
import pandas as pd
import re
import emoji
from nltk.tokenize import word_tokenize
from collections import Counter
from nltk.util import ngrams
from nltk.lm.preprocessing import padded_everygram_pipeline, pad_boundaries
from nltk.lm import MLE

import kagglehub
path = kagglehub.dataset_download("adizafar/large-random-tweets-from-pakistan")

csv_path = glob.glob(path + "/*.csv")[0]

df = pd.read_csv(csv_path, encoding="latin1")

tweets = df['full_text'].astype(str).tolist()
```

```
/Users/firas/miniconda3/envs/default/lib/python3.12/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
```

```
from .autonotebook import tqdm as notebook_tqdm
```

```
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version (1.0.2).
```

```
/var/folders/mn/rxphwjv91f7c4_982kq4kgs80000gn/T/ipykernel_27074/3455254243.py:16: DtypeWarning: Columns (5) have mixed types. Specify dtype option on import or set low_memory=False.
```

```
df = pd.read_csv(csv_path, encoding="latin1")
```

```
In [21]: def clean_tweet(tweet):
    tweet = re.sub(r"http\S+|www\S+", "", tweet) # remove website
    tweet = re.sub(r"@w+", "", tweet) # remove mention
    tweet = re.sub(r"#w+", "", tweet) # remove hashtag
    tweet = re.sub(r"\bRT\b", "", tweet) # remove RT
    tweet = emoji.replace_emoji(tweet, "") # remove emojis
    tweet = re.sub(r"^[A-Za-z\s]", "", tweet) # remove extra
    tweet = re.sub(r"\s+", " ", tweet).strip().lower() # remove extra
    return tweet

cleaned_tweets = [clean_tweet(t) for t in tweets]
tokenized_tweets = [word_tokenize(t) for t in cleaned_tweets if t]
```

```
In [22]: train_data, vocab = padded_everygram_pipeline(2, tokenized_tweets)

lm = MLE(2)
lm.fit(train_data, vocab)
print(len(lm.vocab))
```

85575

```
In [23]: unigram_counts = Counter(word for tweet in tokenized_tweets for word in tweet)
bigram_counts = Counter(pair for tweet in tokenized_tweets for pair in zip(tweet, tweet[1:]))
trigram_counts = Counter(triple for tweet in tokenized_tweets for triple in zip(tweet, tweet[1:2]))

print("Count of unigram 'pakistan':", unigram_counts['pakistan'])
print("Count of bigram 'is/pakistan':", bigram_counts[('is', 'pakistan')])
print("Count of trigram 'a/pakistan is':", trigram_counts[('a', 'pakistan', 'is')])

print("P(pakistan | <s>):", lm.score('pakistan', ['<s>']))
print("P(is | pakistan):", lm.score('is', ['pakistan']))
pakistan_is_count = bigram_counts[('pakistan', 'is')]
probability_a = trigram_counts[('pakistan', 'is', 'a')] / pakistan_is_count
print("P(a | pakistan is):", probability_a)
```

Count of unigram 'pakistan': 12723

Count of bigram 'is/pakistan': 117

Count of trigram 'a/pakistan is': 0

P(pakistan | <s>): 0.00870220590800924

P(is | pakistan): 0.05431109015169378

P(a | pakistan is): 0.10130246020260492

```
In [24]: for seed in range(3):
```

```

generated_tweet = lm.generate(20, text_seed=['<s>'], random_seed=1)
if '</s>' in generated_tweet:
    generated_tweet = generated_tweet[:generated_tweet.index('</s>')]
print("Generated tweet:", ' '.join(generated_tweet))

evaluation_ngrams = ngrams(pad_both_ends(tokenized_tweets[0], n=2),
print("Perplexity:", lm.perplexity(evaluation_ngrams))

```

Generated tweet: to set a fantastic job guys show in open line warriors northern areas of punjab inaugurates western world record straight

Generated tweet: chart looking like curtailing islam has now started being a standing guard of

Generated tweet: woohoo internet

Perplexity: 75.40431421746982

Part2 Task#1

```

In [25]: import glob, string
import pandas as pd
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from collections import Counter
import kagglehub

path = kagglehub.dataset_download("lakshmi25npathi/imdb-dataset-of-50k-reviews")
df = pd.read_csv(glob.glob(path + "/*.csv")[0])
stop = set(stopwords.words("english"))

def preprocess(text):
    text = re.sub(r"<[^>+>", " ", str(text).lower())
    toks = word_tokenize(text)
    return [w for w in toks if w.isalpha() and w not in stop]

df["tokens"] = df["review"].apply(preprocess)

print("Total reviews:", len(df))
print("Avg words/review:", round(df["tokens"].apply(len).mean(), 2))
print("Reviews with 'good':", df["tokens"].apply(lambda t: "good" if "good" in t else "bad").value_counts())
print("Reviews with 'bad':", df["tokens"].apply(lambda t: "bad" if "bad" in t else "good").value_counts())
freq = Counter(w for ts in df["tokens"] for w in ts)
print("Top 10 words:", freq.most_common(10))

```

Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version (1.0.2).

Total reviews: 50000

Avg words/review: 114.42

Reviews with 'good': 18673

Reviews with 'bad': 11600

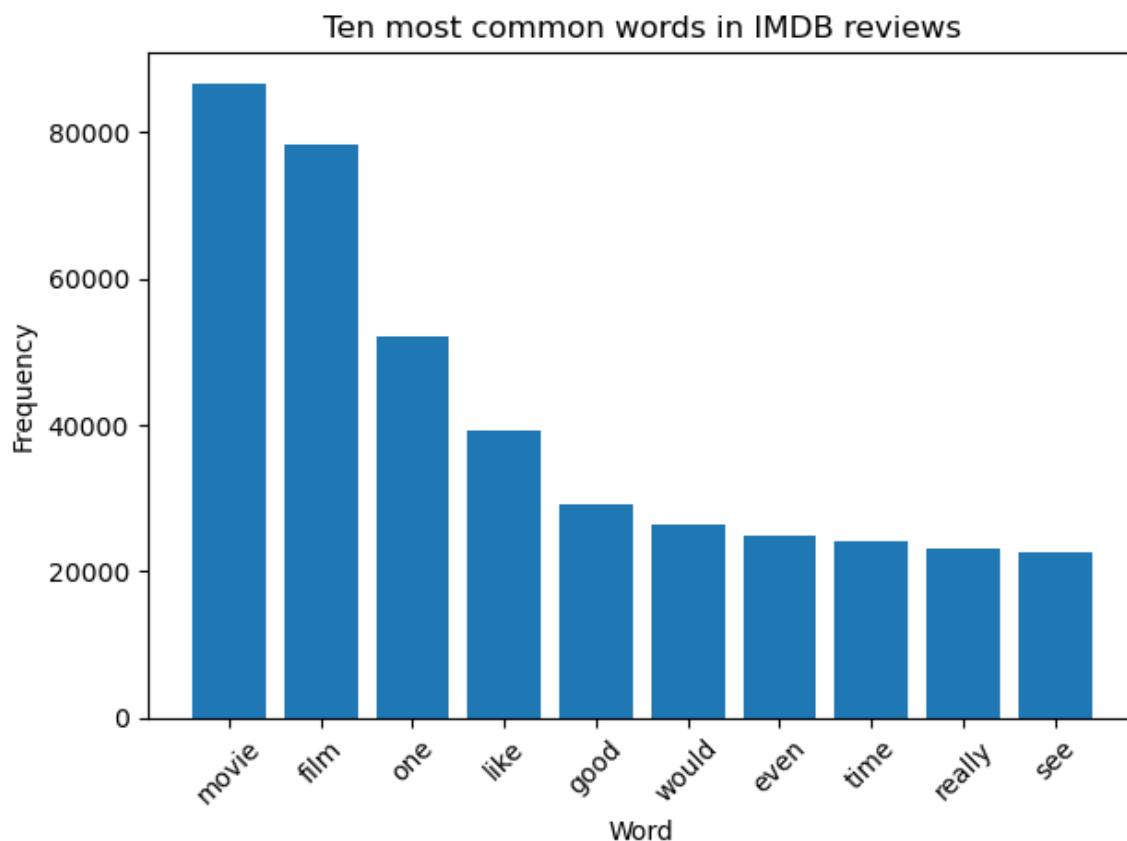
Top 10 words: [('movie', 86537), ('film', 78323), ('one', 52203), ('like', 39232), ('good', 29045), ('would', 26489), ('even', 24786), ('time', 24098), ('really', 23068), ('see', 22717)]

```

In [26]: import matplotlib.pyplot as plt

```

```
words, counts = zip(*freq.most_common(10))
plt.bar(words, counts)
plt.xlabel("Word")
plt.ylabel("Frequency")
plt.title("Ten most common words in IMDB reviews")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Part2 Task2

```
In [27]: from nltk.util import bigrams

path2 = kagglehub.dataset_download("ramjasmaurya/poem-classification")
pdf = pd.concat([pd.read_csv(f) for f in glob.glob(path2 + "/*.csv")])
texts = pdf["Poem"].dropna().astype(str).tolist()
ptoks = [preprocess(t) for t in texts]
pdf2 = pd.DataFrame({"tokens": ptoks})
pdf2["bigrams"] = pdf2["tokens"].apply(lambda t: list(bigrams(t)))
freq2 = Counter(b for lst in pdf2["bigrams"] for b in lst)
print("Top 10 bigrams:", freq2.most_common(10))
```

Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading to the latest version (1.0.2).

Top 10 bigrams: [(('let', 'us'), 11), (('every', 'day'), 9), (('come', 'back'), 8), (('ah', 'yes'), 8), (('first', 'time'), 7), (('best', 'heart'), 6), (('let', 'go'), 6), (('parking', 'lot'), 6), (('give', 'us'), 6), (('still', 'alive'), 5)]

Lab 3 complete